

Find Median from Data Stream

Problem Statement: Implement a class that finds the median from a data stream. The median is the middle value in an ordered integer list. If the size of the list is even, there is no middle value, and the median is the mean of the two middle values.

Implement the MedianFinder class as follows:

- MedianFinder() initializes the MedianFinder object.
- void addNum(int num) adds the integer num to the data structure.
- double findMedian() returns the median of all elements so far. Answers within 10⁻⁵ of the actual answer will be accepted.

Examples

Input : [MedianFinder(), addNum(1), addNum(2), addNum(3), findMedian()]

Output : [null, null, null, null, 2]

Explanation : MedianFinder() initializes the object. addNum(1) adds 1 to the data stream → [1] addNum(2) adds 2 → [1, 2] addNum(3) adds 3 → [1, 2, 3] findMedian() returns 2 as the median of [1, 2, 3]

Input : [MedianFinder(), addNum(1), addNum(6), findMedian(), addNum(3), findMedian()]

Output : [null, null, null, 3.5, null, 3]

Explanation : MedianFinder() initializes the object. addNum(1) → [1] addNum(6) → [1, 6] findMedian() returns $(1+6)/2 = 3.5$ addNum(3) → [1, 3, 6] findMedian() returns 3 as the median

Brute Force Approach

Algorithm

We maintain a list of all numbers inserted so far. Every time we want the median, we sort the entire list and return either the middle element (odd size) or the average of the two middle elements (even size). This is simple to implement but inefficient for large inputs.

- Create an empty list to store all incoming numbers.
- On calling addNum, insert the number into the list.
- On calling findMedian:
 - Sort the list.
 - If size is odd, return the middle element.
 - If size is even, return average of the two middle elements.

```

nums = []

mf.addNum(1)    nums [ 1 ]

mf.addNum(2)    nums [ 1 2 ]

mf.addNum(3)    nums [ 1 2 3 ]

mf.findMedian()
    ↪ nums = [ 1 2 3 ] {sort array}
                        count = 3 → odd

ans = nums[3/2] = nums[1] = 2

```

Code

```

import java.util.*;

class MedianFinder {
    // List to store all the numbers
    private List<Integer> nums;

    // Constructor to initialize the object
    public MedianFinder() {
        nums = new ArrayList<>();
    }

    // Function to add a number to the list
    public void addNum(int num) {
        // Add number to the list
        nums.add(num);
    }

    // Function to return the median
    public double findMedian() {
        // Sort the list
        Collections.sort(nums);

        // Get size
        int n = nums.size();

        // If odd, return middle
        if (n % 2 == 1) {
            return nums.get(n / 2);
        }

        // If even, return average of two middle
    }
}

```

```

        return (nums.get(n / 2 - 1) + nums.get(n / 2)) / 2.0;
    }
}

// Driver code
class Main {
    public static void main(String[] args) {
        MedianFinder mf = new MedianFinder();

        mf.addNum(1);
        mf.addNum(2);
        mf.addNum(3);

        System.out.println(mf.findMedian()); // Output: 2.0
    }
}

```

Complexity Analysis

Time Complexity: $O(N \log N)$, Every call to `findMedian()` sorts the list of N numbers, which takes $O(N \log N)$ time. Adding a number is $O(1)$. Hence, the median query is the bottleneck.

Space Complexity: $O(N)$, We store all elements in a list, which uses linear space.

Optimal Approach

Algorithm

To efficiently find the median after each insertion, we can use two heaps: A max-heap to store the smaller half of the numbers. A min-heap to store the larger half of the numbers. The max-heap gives access to the largest element of the smaller half, and the min-heap gives access to the smallest element of the larger half. This way, the median will always be either: The top of the max-heap (if total size is odd), or The average of the tops of both heaps (if even). We maintain the following invariants: The size of the max-heap is either equal to or 1 greater than the min-heap. All elements in the max-heap are less than or equal to those in the min-heap.

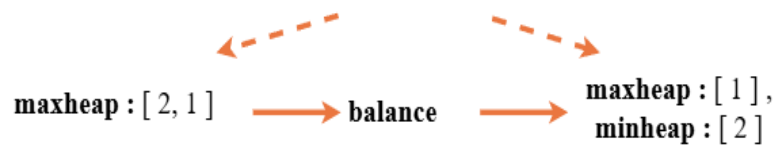
- Initialize two heaps: a max-heap for the smaller half and a min-heap for the larger half.
- When inserting a new number:
 - If it's less than or equal to the top of the max-heap, insert it into the max-heap.
 - Otherwise, insert it into the min-heap.
- Balance the heaps if their sizes differ by more than 1.
- To find the median:
 - If both heaps are of equal size, return the average of the tops of both heaps.
 - Otherwise, return the top of the heap that has one extra element.

maxheap : []
minheap : []

addNum(1)

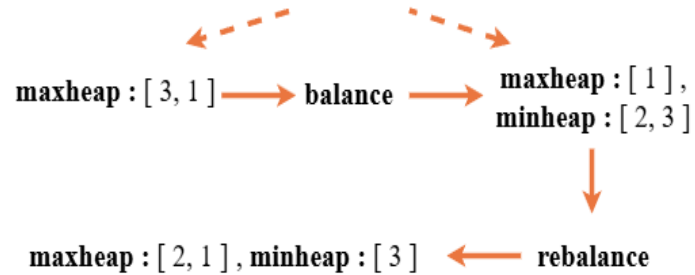


addNum(2)



findMedian()
 $= (1 + 2) / 2 = 1.5$

addNum(3)



findMedian () $\rightarrow 2$

Code

```
import java.util.*;

class MedianFinder {
    // Max-heap for smaller half
    private PriorityQueue<Integer> maxHeap;

    // Min-heap for larger half
    private PriorityQueue<Integer> minHeap;

    // Constructor
    public MedianFinder() {
        maxHeap = new PriorityQueue<>(Collections.reverseOrder());
        minHeap = new PriorityQueue<>();
    }

    // Function to add a number to the data structure
    public void addNum(int num) {
        // Step 1: Add to maxHeap
        maxHeap.offer(num);

        // Step 2: Move the largest of maxHeap to minHeap
        minHeap.offer(maxHeap.poll());

        // Step 3: Balance the heaps if needed
        if (minHeap.size() > maxHeap.size()) {
            maxHeap.offer(minHeap.poll());
        }
    }

    // Function to return the current median
    public double findMedian() {
        // If equal sizes, return average
        if (maxHeap.size() == minHeap.size()) {
            return (maxHeap.peek() + minHeap.peek()) / 2.0;
        }

        // Otherwise, return top of maxHeap
        return maxHeap.peek();
    }
}

// Driver code
class Main {
    public static void main(String[] args) {
        MedianFinder mf = new MedianFinder();
        mf.addNum(1);
        mf.addNum(2);
        System.out.println(mf.findMedian()); // 1.5
        mf.addNum(3);
        System.out.println(mf.findMedian()); // 2
    }
}
```

Complexity Analysis

Time Complexity: $O(N \log N + M)$, Because each call to addNum() takes $O(\log N)$, and each call to findMedian() is $O(1)$.

Space Complexity: $O(N)$, We store all the numbers across two heaps. The max-heap stores the smaller half of numbers (up to $N/2$) and the min-heap stores the larger half (up to $N/2$) So, the total space used is $O(N)$, where N is the number of elements added to the data structure.